26. Disks, Partitions, Filesystems, mount

26. Disks, Partitions, Filesystems, mount

Phase: Phase 8 — Packages, disks, storage
Estimated time: 2 days
Slug: `26-disks-fs`

Learning objectives

- List block devices
- Understand partitions vs filesystems
- Mount/umount carefully
- Check free space and inodes

Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics. Type every command yourself — muscle memory beats skimming.

Why this matters

“No space left on device” is one of the most common production failures, and it has a confusing variant: the disk shows free space but you still cannot create files. That second case means you ran out of inodes, the metadata slots that each file consumes. To handle storage you need to read block devices, partitions, filesystems, mount points, and both kinds of capacity: bytes and inodes.

Core concepts

Term	Meaning
Block device	Raw storage: `/dev/sda`, `/dev/nvme0n1`, `/dev/vda`
Partition	A slice of a disk: `/dev/sda1`
Filesystem	Structure on the partition: ext4, xfs, btrfs
Mount point	Directory where a filesystem appears: `/`, `/data`
UUID	Stable filesystem identifier
Inode	Metadata record for one file or directory
Block usage	Bytes used (`df -h`)
Inode usage	Files used (`df -i`)

Tools:

Command	Shows
`lsblk -f`	Devices, filesystem types, UUIDs, mounts
`df -h`	Space used per mounted filesystem
`df -i`	Inodes used per mounted filesystem
`du -sh dir`	Space consumed by a directory
`findmnt`	Mount tree
`blkid`	UUIDs and types
`fdisk -l` / `parted`	Partition tables
`mkfs.ext4`, `mount`, `umount`	Create, attach, detach

Layers from disk to path:

```
physical disk   /dev/vdb
      |
partition       /dev/vdb1          (fdisk / parted)
      |
filesystem      ext4, UUID=abcd... (mkfs.ext4)
      |
mount point     /data              (mount /dev/vdb1 /data)
      |
files           /data/app/log.txt  (each file = 1 inode + data blocks)

Full if EITHER bytes run out (df -h) OR inodes run out (df -i)
```

Worked examples

Overview of devices:

```
lsblk -f
```

Sample output:

```
NAME   FSTYPE FSVER LABEL UUID                                 FSAVAIL FSUSE%
MOUNTPOINTS
vda
|-vda1 ext4   1.0         1c2d3e4f-0000-4a5b-9c8d-aaaaaaaaaaaa   12.4G    38% /
vdb
`-vdb1 ext4   1.0         9f8e7d6c-1111-4b3a-8d7c-bbbbbbbbbbbb    9.1G     1% /
data
```

Space and inodes:

```
df -h /
df -i /
```

Sample output:

```
Filesystem      Size  Used Avail Use% Mounted on
/dev/vda1        20G  7.6G   12G  39% /
Filesystem      Inodes  IUsed   IFree IUse% Mounted on
/dev/vda1      1310720 204118 1106602   16% /
```

Find what is using space:

```
sudo du -xh / --max-depth=1 2>/dev/null | sort -h | tail -4
sudo find /var -xdev -type f -size +100M -exec ls -lh {} +
```

Sample output:

```
1.2G    /usr
2.4G    /var
7.0G    /
```

Prepare a new disk (destructive, lab VM only):

```
sudo parted -s /dev/vdb mklabel gpt mkpart primary ext4 1MiB 100%
sudo mkfs.ext4 /dev/vdb1
sudo mkdir -p /data
sudo mount /dev/vdb1 /data
findmnt /data
```

Sample output:

```
TARGET SOURCE    FSTYPE OPTIONS
/data  /dev/vdb1 ext4   rw,relatime
```

Run out of inodes on purpose in a small loop filesystem:

```
truncate -s 20M /tmp/small.img
mkfs.ext4 -N 512 -q /tmp/small.img
sudo mount -o loop /tmp/small.img /mnt
sudo bash -c 'for i in $(seq 1 600); do : > /mnt/f$i || break; done'
df -i /mnt
```

Sample output:

```
touch: cannot touch ... No space left on device
Filesystem     Inodes IUsed IFree IUse% Mounted on
/dev/loop0        512   512     0  100% /mnt
```

Helper from the exercise: `inode_check_flag() { echo "-i"; }` returns the flag you pass to `df` (bash echo only treats -n, -e, -E as options).

Mini-lab

1. Run `lsblk -f` and identify the root device and its UUID.
2. Compare `df -h /` and `df -i /`.
3. Use `du -xh /var --max-depth=1 | sort -h` to find the biggest directory.
4. Create a 50M loop image, format it with ext4, and mount it on `/mnt`.
5. Fill it with `fallocate -l 40M /mnt/big`, check `df -h /mnt`.

6. Create many tiny files and watch `df -i /mnt` climb.
7. Unmount with `sudo umount /mnt`; if busy, find the user with `lsof +f -- /mnt` or `fuser -vm /mnt`.
8. Delete a large file held open by a process and see that space returns only after the process closes it (`lsof +L1`).

Decision guide / failure modes

Symptom	Cause	Fix
`No space left`, `df -h` shows free	Inodes exhausted	`df -i`, delete tiny files, recreate fs with more inodes
`df` full but `du` totals smaller	Deleted file still open	`lsof +L1`, restart the process
`target is busy` on umount	Process cwd or open file inside	`fuser -vm`, `lsof`
Mount disappears after reboot	Not in fstab	Add UUID entry
Read-only filesystem	Errors detected	`dmesg`, `fsck` after unmount
Wrong disk formatted	Used wrong device name	Always confirm with `lsblk` first

Before any `mkfs`, `dd`, or `parted`, read `lsblk` twice and confirm the device by size and UUID. For capacity questions always ask both: bytes with `df -h`, inodes with `df -i`.

Common mistakes

- mkfs on the wrong disk
- umount busy filesystems without checking
- Ignoring inode usage

Exercises

Work in `exercises/26-disks-fs/`. Read that folder's `README.md` fully before coding.

```
cd exercises/26-disks-fs
chmod +x test.sh exercise.sh
./test.sh
```

What you will build

Implement inode_check_flag in exercise.sh. It takes no arguments and prints the option you pass to df to report inode usage instead of block usage. Print only the flag, including its leading dash. Be careful when testing by hand: echo with a leading dash can be read as an option by some shells, so confirm that your function prints the text exactly (printf '%s' is

a safe choice). Afterwards run df -i / and df -h / on a real host and explain in one sentence why the two can disagree.

Study loop

- **A (Drill):** make `./test.sh` pass
- **B (Apply):** run 2–3 lab commands from this lesson on a real Linux host; paste notable output into your notes
- **C (Challenge):** rewrite the exercise from memory the next day

Do not open `solutions/26-disks-fs/` until you have an honest failing attempt.

Self-check

Try to answer without scrolling up:

1. lsblk shows what?
2. Why df -i?
3. mount purpose?

Further reading

- https://man7.org/linux/man-pages/man8/lsblk.8.html